

שפות תכנות – תרגיל 2

תאריך הגשה: 19.5.2026

הוראות הגשה: ההגשה בזוגות או ביחידים דרך אתר למדה. כל זוג נדרש לחשוב, לפתור ולכתוב את התרגיל בעצמו. יש לקרוא הוראות אלא בקפידה. הגשה שלא על פי הוראות אלה תוביל להורדת ניקוד. שימו לב שרק אחד מבני הזוג מגיש.

בתרגילי תכנות יש להגיש קובץ sml. עם השם של הפונקציה שאותה יש לממש (לדוגמא עבור הפונקציה insert_at שאתם צריכים לממש הקובץ שבו נמצאת הפונקציה יקרא insert_at.sml). יש להגיש קובץ עבור כל פונקציה שנדרשתם לכתוב גם אם הם באותה שאלה בסעיפים שונים. במידה ויש הגדרות של טיפוסים בשאלה צריך להוסיף אותם לקובץ, אין להדפיס למסך שום דבר שאינו נדרש בתרגיל עצמו. מותר ואף מומלץ להשתמש בפונקציות עזר שהפונקציה שאתם כותבים תקרא להם (פונקציות עזר יהיו באותו קובץ של הפונקציה הראשית).

חומר עזר מומלץ: כדאי להבין היטב את הרצאות ותרגולים.

התקנת סביבת עבודה בפעם הראשונה, מומלץ לעקוב אחרי ההנחיות שבתרגול

מה להגיש: רשימה של הקבצים שיש להגיש:

Insert_at.sml

insert_none_at.sml

add_to_search_tree.sml

table.sml

קובץ ת.ז של כל אחד מהמגשים (כל מגיש בשורה בנפרד) – id.txt

כל הקבצים צריכים להיות בקובץ zip אחד בשם: ex2.zip.

1. חימום - הכנסת ערך לרשימה במיקום מסויים:

א. כתבו פונקציה בשם `insert_at` המקבלת איבר אינדקס ורשימה ומכניסה את האיבר לרשימה באינדקס הנכון.

דוגמאות להרצה:

```
- insert_at #"e" 1 ["a", "b", "c"];
val it = ["a",#"e",#"b",#"c"] : char list
- insert_at #"e" 3 ["a", "b", "c"];
val it = ["a",#"b",#"c",#"e"] : char list
- insert_at #"e" 0 ["a", "b", "c"];
val it = ["e",#"a",#"b",#"c"] : char list
```

הנחת קלט: אנחנו מניחים שברשימה n יש לפחות n איברים, כאשר n הוא האינדקס הנבחר להכנסה.

ב. במפעל מסוים רצו ליצור פונקציה שמכניסה אחרי `None` במקומות מסוימים ברשימות שלהם. כתבו פונקציה בשם `insert_none_at` שתקבל אינדקס להכנסה ורשימה ותכניס את הערך `"None"` ברשימה באינדקס שהתקבל. שימו לב הפונקציה החדשה צריכה לקרוא לפונקציה מסעיף א.

דוגמאות להרצה:

```
- insert_none_at 2 ["a", "b", "c"];
val it = ["a","b","None","c"] : string list
- insert_none_at 0 ["Not None", "Not None", "Not None", "Not None"];
val it = ["None","Not None","Not None","Not None","Not None"] : string list
```

הנחת קלט: אנחנו מניחים שברשימה n יש לפחות n איברים, כאשר n הוא האינדקס הנבחר להכנסה.

2. עץ חיפוש בינארי

תזכורת: בתרגול ראינו שאפשר להגדיר עץ בינארי בצורה הבאה:

```
datatype 'a binary_tree =
  Empty
  | Node of 'a * 'a binary_tree * 'a binary_tree;
```

א. כתבו פונקציה `add_to_search_tree` המקבלת עץ חיפוש בינארי של מספרים שלמים ומספר שלם, ומחזירה עץ חיפוש חדש שמתקבל מהוספת המספר לעץ החיפוש הבינארי שהתקבל בקלט. עץ חיפוש בינארי הוא עץ בינארי כך שכל הערכים הקטנים מהערך של הקודקוד נמצאים בתת העץ השמאלי של הקודקוד וכל הקודקודים בעלי ערך גבוה יותר נמצאים בתת העץ הימני של הקודקוד.

דוגמא להרצה:

```
- val tree0 = Empty;
val tree0 = Empty : 'a tree
- val tree1 = add_to_search_tree tree0 10;
val tree1 = Node (10,Empty,Empty) : int tree
- val tree2 = add_to_search_tree tree1 5;
val tree2 = Node (10,Node (5,Empty,Empty),Empty) : int tree
- val tree3 = add_to_search_tree tree2 15;
val tree3 = Node (10,Node (5,Empty,Empty),Node (15,Empty,Empty)) : int tree
```

הנחה על הקלט: - בעץ החיפוש הבינארי לא יהיו שני מספרים בעלי ערך זהה בכל קריאה לפונקציה.

- המספר שמתקבל לפונקציה לא יהיה שווה לאחד מאיברי העץ.

ב. מה הייתם משנים בפונקציה שכתבתם בסעיף א כדי שתתמוך גם בעץ חיפוש בינארי שכל איבריו הם מספרים של נקודה צפה (float).
הערה: תשובה לסעיף ב צריכה להתווסף לקובץ `sml` בהערה.

3. לוגיקה וטבלאות אמת

נגדיר את ה-variant type הבא שיעזור לנו לייצג ביטויים לוגיים עם משתנים בוליאניים:

```
- datatype expr = Var of string | Not of expr | And of expr * expr |
  Or of expr = * expr;
datatype expr
  = And of expr * expr
  | Not of expr
  | Or of expr * expr
  | Var of string
```

ניתן לייצג ביטויים לוגיים בעזרת ה-type החדש. לדוגמא הביטוי הלוגי $(a \vee b) \wedge (a \wedge b)$, ייוצג באופן הבא:

```
- (And (Or (Var "a", Var "b"), And (Var "a", Var "b")));
val it = And (Or (Var "a",Var "b"),And (Var "a",Var "b")) : expr
```

הגדירו פונקציה table שבהינתן רשימה של n משתנים בוליאניים וביטוי לוגי (ללא משתנים נוספים חוץ מהשניים שהוגדרו לפני) מחזירה את טבלת האמת של הביטוי הלוגי. טבלת האמת תודפס למסך רשימה שבכל איבר יש $n+1$ ערכים בוליאניים. n ערכים של השמות למשתנים והערך האחרון של הביטוי הלוגי בהינתן הערכים למשתנים.

דוגמא להרצה:

```
- table ["a", "b"] (And (Or (Var "a", Var "b"), And (Var "a", Var "b")));
val it =
  [([("b",true),("a",true)],true),([("b",false),("a",true)],false),
    ([("b",true),("a",false)],false),([("b",false),("a",false)],false)]
  : ((string * bool) list * bool) list
```

```
- table ["a", "b", "c"] (Or(And(Var "a", Var "b"), Var "c"));
val it =
  [([("c",true),("b",true),("a",true)],true),
    ([("c",false),("b",true),("a",true)],true),
    ([("c",true),("b",false),("a",true)],true),
    ([("c",false),("b",false),("a",true)],false),
    ([("c",true),("b",true),("a",false)],true),
    ([("c",false),("b",true),("a",false)],false),
    ([("c",true),("b",false),("a",false)],true),
    ([("c",false),("b",false),("a",false)],false)]
  : ((string * bool) list * bool) list
```

לשם האחידות נגדיר את הסדר של האיברים כך:

$a = \text{true}, b = \text{true}$

$a = \text{true}, b = \text{false}$

$a = \text{false}, b = \text{true}$

$a = \text{false}, b = \text{false}$

זה הסדר שמופיע בדוגמא להרצה.

סדר האיברים יהיה עבור האיבר הראשון יופיע כל האפשרויות שלו ל-true ולאחר מכן של false. עבור שאר המשתנים חוץ מהראשון הסדר יהיה הופעה של כל האיברים שבהם האיבר הראשון (מתוך ה- $n-1$ איברים הוא true ואחר כך כל האיברים בהם הוא false כך יהיה סדר הטבלה באופן רקורסיבי (כמו שמופיע בדוגמא להרצה).